

Análisis Individual

[Categoría: Arquitectura]

- **Ubicación / Componente:** `src/App.jsx`, `src/components/Nav.jsx`
 - **Problema Identificado:** `Header` y `Nav` se renderizan de forma incondicional en `App.jsx` para todas las rutas, incluida `/login`. Esto significa que el menú "Dashboard / Clientes" queda visible y clickeable incluso cuando no hay sesión iniciada. Al intentar navegar desde ahí a una ruta protegida sin `admin`, `RutaProtegida` redirige de inmediato a `/login`, generando un rebote confuso y, si llega a dispararse algún efecto antes de la redirección, llamadas de red innecesarias.
 - **Impacto:** Confunde al usuario no autenticado ofreciéndole navegación a secciones a las que no puede acceder, y genera comportamiento de redirección brusco en vez de un login limpio.
 - **Mejora a Implementar:** Condicionar el renderizado de `Nav` (y el bloque de usuario en `Header`) a la existencia de `admin` desde `useAutorizaciones()`, o definir un layout separado para `/login` que no incluya el menú de navegación interno.
-

[Categoría: Deuda Técnica]

- **Ubicación / Componente:** `src/components/Header.jsx` (botón "Cerrar Sesión"), `src/components/FormCliente.jsx` (botón "Guardar Cliente"), `src/pages/DetalleCliente.jsx` (botón "Eliminar Cliente"), `src/pages/ListaClientes.jsx` (link "Ver Ficha Completa")
 - **Problema Identificado:** Todos los botones y acciones de la interfaz usan únicamente texto plano, sin íconos que refuercen visualmente la acción (salir, guardar, eliminar, ver detalle).
 - **Impacto:** Menor atractivo visual y menor velocidad de reconocimiento de acciones por parte del usuario, especialmente en una interfaz de gestión donde los íconos ayudan a escanear rápido la tabla de clientes.
 - **Mejora a Implementar:** El proyecto ya depende de `react-bootstrap`; incorporar una librería de íconos compatible (por ejemplo `react-icons`) y añadir un ícono junto al texto en cada botón clave (logout, guardar, eliminar, ver ficha).
-

[Categoría: Arquitectura]

- **Ubicación / Componente:** `src/pages/ListaClientes.jsx` (incluye `<FormCliente />` inline)
 - **Problema Identificado:** El formulario de alta de clientes (`FormCliente`) se renderiza dentro de la misma página que el listado y buscador (`ListaClientes`), mezclando dos responsabilidades de UI distintas (creación y consulta) en una sola vista.
 - **Impacto:** La página se sobrecarga visualmente, dificulta el mantenimiento del componente y limita la navegación (no se puede enlazar directamente a "crear cliente" ni gestionar el estado de cada flujo de forma independiente).
 - **Mejora a Implementar:** Separar en dos rutas distintas, por ejemplo `/clientes` (listado) y `/clientes/nuevo` (formulario), enlazadas mediante `Link/useNavigate`, siguiendo el mismo patrón ya usado para `/clientes/:id`.
-

[Categoría: React]

- **Ubicación / Componente:** `src/pages/ListaClientes.jsx` (`clientesFiltrados`)
 - **Problema Identificado:** El filtro de búsqueda solo evalúa `cliente.name.lastname` y `cliente.address.city`. Campos como nombre de pila, email, teléfono o ID quedan fuera del buscador pese a mostrarse en la tabla.
 - **Impacto:** El usuario espera poder buscar por cualquier dato visible en la tabla; la búsqueda parcial actual da resultados vacíos o incompletos ante términos válidos (por ejemplo, buscar por email no devuelve nada).
 - **Mejora a Implementar:** Extender la condición del `.filter()` para incluir `name.firstname`, `email` y `phone` (con las validaciones de `?.` correspondientes), por ejemplo combinando todos los campos relevantes en un solo string normalizado antes de aplicar `.includes()`.
-

[Categoría: Estado]

- **Ubicación / Componente:** `src/components/FormCliente.jsx` y `src/pages/ListaClientes.jsx`
- **Problema Identificado:** `FormCliente` crea el cliente vía `clientesService.crearCliente` pero no existe ningún mecanismo (callback, contexto o store compartido) para notificar a `ListaClientes` del nuevo registro.
- **Impacto:** Tras un alta exitosa, la tabla y el contador "Clientes encontrados" no se actualizan; el usuario debe recargar la página para intentar verlo reflejado (y como FakeStoreAPI no persiste datos, ni siquiera así aparecerá).
- **Mejora a Implementar:** Levantar el estado (`lifting state up`): pasar una función `onClienteCreado(nuevoCliente)` desde `ListaClientes` hacia

`FormCliente` para insertar el nuevo cliente en el arreglo local (`setClientes(prev => [...prev, nuevoCliente])`) apenas se confirma la creación.

[Categoría: Deuda Técnica]

- **Ubicación / Componente:** `src/css/*.css` (estilos globales de la aplicación)
 - **Problema Identificado:** Se utiliza un único color primario para toda la interfaz, sin una paleta definida que diferencie jerarquías visuales (acciones primarias, secundarias, alertas, secciones).
 - **Impacto:** La interfaz resulta plana y dificulta distinguir visualmente distintos tipos de acción o contenido (por ejemplo, un botón de eliminar debería destacarse del de guardar).
 - **Mejora a Implementar:** Definir una paleta de colores mediante variables CSS (`:root { --color-primary: ...; --color-danger: ...; --color-bg-section: ...; }`) y aplicarla de forma consistente en los distintos archivos CSS del proyecto en lugar de repetir valores hardcodeados.
-

[Categoría: Arquitectura]

- **Ubicación / Componente:** `src/pages/Login.jsx`, `src/services/autorizacionesServices.js`
 - **Problema Identificado:** El login exige seleccionar manualmente un `sector` (Soporte/Gerencia) además de email y contraseña, y la función `login()` filtra por los tres campos exactos. Esto es redundante porque cada usuario ya tiene un `sector` fijo asociado en el arreglo `usuarios`.
 - **Impacto:** El usuario debe recordar/adivinar su propio sector para poder ingresar; si selecciona el sector incorrecto, el login falla con credenciales por lo demás correctas, generando fricción y errores evitables (`alert('Verifique los datos')`) sin causa real.
 - **Mejora a Implementar:** Eliminar el campo "Sector" del formulario. Modificar `login(email, password)` para buscar el usuario solo por email y contraseña, y devolver el `sector` ya almacenado en el registro encontrado, asignándolo automáticamente al `admin` del contexto.
-

[Categoría: Deuda Técnica]

- **Ubicación / Componente:** `src/css/*.css` (secciones: dashboard, listado, detalle, login, header, nav, footer)

- **Problema Identificado:** No hay una diferenciación de color de fondo entre los distintos apartados de la página (Dashboard, Clientes, Detalle, Login), lo que suma a la falta de jerarquía visual ya señalada con el color primario único.
- **Impacto:** Todas las secciones se perciben visualmente iguales, dificultando al usuario ubicarse rápidamente dentro de la aplicación sin depender exclusivamente del texto del menú.
- **Mejora a Implementar:** Asignar fondos distintivos (sutiles, dentro de la misma paleta de variables CSS propuesta) por sección mediante una clase de contenedor específica por página (`.dashboard`, `.clientes-container`, `.detalle-cliente`, `.login-container`, todas ya existentes) para reforzar la identidad visual de cada vista.